

JavaScript

Aula 1: tipos, operadores, arrays
e funções



O que vamos ver hoje

01 Tipos primitivos

02 Imprimindo no console

03 Operadores aritméticos

04 Operadores de atribuição

05 Comparação e lógicos

06 Template strings

07 Arrays

08 Funções declaradas



01 Tipos primitivos

São os tipos básicos de dado que o JavaScript já entende sem você criar nada.

- **string** — texto entre aspas
- **number** — inteiros ou decimais
- **boolean** — só true ou false
- **undefined** — criada, sem valor
- **null** — vazio de propósito

tipos.js

```
let nome = "Ana";
let idade = 20;
let media = 8.5;
let matriculado = true;
let apelido = null;
let nota; // undefined

console.log(typeof nome); // "string"
console.log(typeof idade); // "number"
```



01 Tipos: 3 armadilhas

Erros que todo iniciante comete no primeiro dia.

no console

```
"20" + 5      // "205"   (juntou texto!)
"20" - 5      // 15      (converteu)
Number("20") + 5 // 25   (jeito certo)

typeof null    // "object"
typeof undefined // "undefined"
```

+ com texto vira texto

Se um dos lados é string, o + concatena em vez de somar. Use Number() para converter.

undefined ≠ null

undefined é "ninguém colocou valor ainda". null é "eu coloquei vazio de propósito".

typeof null dá "object"

É um bug histórico da linguagem, mantido por compatibilidade. Decore e siga em frente.



02 Imprimindo no console

`console.log()` é como o programa conversa com você. Abra o navegador e aperte F12.

aula.js

```
let nome = "Ana";  
let nota = 8.5;  
  
console.log("Boas-vindas!");  
console.log(nome);  
console.log("Nota:", nota);  
console.log(nome, "tirou", nota);
```

Console

```
Boas-vindas!  
Ana  
Nota: 8.5  
Ana tirou 8.5
```

Vírgula, não +

Separando com vírgula o JavaScript imprime tudo com um espaço e sem misturar os tipos.





Suas variáveis

Crie três variáveis sobre você e imprima as três em uma única linha no console.

- nome deve ser uma string
- idade deve ser um number
- estudante deve ser um boolean
- use uma única chamada de console.log com vírgulas
- confira o tipo de cada uma com typeof

complete o código

```
let nome = ____;  
let idade = ____;  
let estudante = ____;  
  
console.log(____, ____, ____);  
  
console.log(typeof idade);
```



03

Operadores aritméticos

A calculadora do JavaScript. Todos devolvem um number.

+

Soma

-

Subtração

*

Multiplicação

/

Divisão

%

Resto

**

Potência

lanchonete.js

```
let precoLanche = 18;
let precoSuco = 7;

console.log(precoLanche + precoSuco); // 25
console.log(precoLanche * 3);         // 54
console.log(precoLanche / 4);         // 4.5
console.log(17 % 2);                   // 1 (sobra)
console.log(2 ** 10);                  // 1024
```

% é o resto da divisão

17 % 2 é 1, então 17 é ímpar. É assim que se testa par ou ímpar: `numero % 2 === 0`.



04

Operadores de atribuição

Atalhos para atualizar o valor que a variável já tem.

jogo.js

```
let saldo = 100;    // = atribui o valor
```

```
saldo += 50;        // = saldo + 50 -> 150
```

```
saldo -= 20;        // 130
```

```
saldo *= 2;         // 260
```

```
saldo /= 4;         // 65
```

```
let vidas = 3;
```

```
vidas++;           // 4   (soma 1)
```

```
vidas--;           // 3   (tira 1)
```

x += 5 é x = x + 5

O operador lê o valor atual, faz a conta e guarda de volta na mesma variável.

++ e -- andam de 1 em 1

Muito usados em contadores e loops: vidas--, pontos++, tentativas++.

= não é comparação

Um sinal de igual atribui. Para comparar você usa === (próximo slide).





Carrinho de compras

Você comprou 3 cadernos de R\$ 12,90 e 2 canetas de R\$ 3,50. Calcule o total e aplique 10% de desconto.

- use * para o subtotal de cada item
- use + para somar os subtotais
- aplique o desconto com *= 0.9
- imprima o total antes e depois do desconto

complete o código

```
let cadernos = 3 * 12.90;  
let canetas = 2 * 3.50;  
let total = ____;  
  
console.log(total);  
  
total *= 0.9;    // 10% off  
console.log(total);
```



05 Operadores de comparação

Toda comparação devolve um boolean: true ou false.

comparacao.js

```
let nota = 7;

nota > 6      // true
nota >= 7     // true
nota < 6      // false
nota !== 10   // true  (diferente)

"7" == 7      // true   converte o tipo
"7" === 7     // false  compara o tipo
```

Erro comum: == no lugar de ===

O == tenta converter os tipos antes de comparar, então "7" == 7 dá true e você descobre o bug muito depois. Use === e !== sempre.

Regra prática

=== compara valor E tipo. É o que você quer em 99% dos casos.



05

Operadores lógicos

Servem para combinar duas ou mais comparações em uma única resposta.

&& E

as duas condições precisam ser true

|| OU

basta uma das condições ser true

! NÃO

inverte o resultado

aprovacao.js

```
let nota = 7;
let presenca = 80;

// && precisa dos DOIS
nota >= 6 && presenca >= 75    // true
nota >= 6 && presenca >= 90    // false

// || basta UM      |      ! inverte
nota >= 9 || presenca >= 75    // true
!(nota >= 6)                  // false
```

Leia em voz alta

"nota maior ou igual a 6 E presença maior ou igual a 75". Se a frase faz sentido em português, o código está certo.





Aprovado ou não

Um aluno é aprovado se tiver nota maior ou igual a 6 E presença maior ou igual a 75%.

- crie as variáveis nota e presença
- guarde o resultado em aprovado usando &&
- imprima o boolean no console
- teste com nota 5 e presença 90 e veja o que muda

complete o código

```
let nota = 7.5;  
let presença = 80;  
  
let aprovado = ____ && ____;  
  
console.log(aprovado);
```



06

Template strings

Monte textos com variáveis dentro usando acento grave (`) e `\${ }`.

jeito antigo

```
// antes: concatenação
let msg = "Ola, " + nome +
  "! Sua nota foi " + nota + ".";

// muitos + e fácil errar espaço
```

jeito bom

```
// agora: template string
let msg = `Ola, ${nome}!
  Sua nota foi ${nota}.`;

// aceita conta: ${nota * 2}
```

` não é '

Fica na tecla antes do 1, no teclado ABNT.

\${ } aceita expressão

Cabe soma, multiplicação e até chamar função.

Quebra linha de graça

A quebra que você escrever é preservada.



07

Arrays: listas de valores

Um array guarda vários valores em uma única variável, na ordem em que você colocou.

materias.js

```
let materias = ["Matematica", "Portugues",  
               "Historia"];  
  
materias[0]      // "Matematica"  
materias[2]      // "Historia"  
materias.length  // 3  
  
materias.push("Fisica");    // adiciona no fim  
materias.pop();             // remove do fim  
materias.indexOf("Portugues"); // 1
```

O índice começa em 0

O primeiro item é [0], o segundo é [1]. Num array de 3 itens, o último é [2].

length é a quantidade

Não é o último índice. O último índice é sempre length - 1.

push, pop, indexOf

push adiciona no fim, pop remove do fim, indexOf devolve a posição ou -1 se não achar.



07

Percorrendo um array

Duas formas de visitar item por item. Escolha a que deixa o código mais claro.

for clássico

```
let notas = [8, 6.5, 9, 7];

for (let i = 0; i < notas.length; i++) {
  console.log(`Nota ${i + 1}: ${notas[i]}`);
}

// use quando precisar do índice
```

`i < notas.length`, nunca `<=`

Com `<=` você tenta acessar uma posição que não existe e recebe `undefined`.

for...of

```
let notas = [8, 6.5, 9, 7];

for (let nota of notas) {
  console.log(nota);
}

// use quando só quer o valor
```

Somando dentro do loop

Crie `soma = 0` antes do `for` e use `soma += nota` dentro. No fim, divida por `notas.length`.





Média das notas

Com o array [8, 6.5, 9, 7]: imprima cada nota, some todas e mostre a média com template string.

- percorra com for...of
- acumule em soma usando +=
- média é soma dividida por notas.length
- imprima com `A média é \${media}`

complete o código

```
let notas = [8, 6.5, 9, 7];
let soma = 0;

for (let n of notas) {
  soma ____;
}

let media = ____ / ____;
console.log(`A media e ${media}`);
```



08

Funções declaradas

Um bloco de código com nome, que você escreve uma vez e usa quantas vezes quiser.

funcoes.js

```
function calcularMedia(n1, n2) {  
  let media = (n1 + n2) / 2;  
  return media;  
}  
  
let resultado = calcularMedia(8, 6);  
console.log(`Media: ${resultado}`); // 7  
  
calcularMedia(10, 5); // 7.5 reaproveitando
```

function nome(...)

A palavra function, o nome da função e os parâmetros entre parênteses.

Parâmetros são apelidos

n1 e n2 só existem dentro da função. Na chamada você passa os valores reais (argumentos).

return devolve o valor

Sem return a função executa mas não entrega nada. E o código depois do return não roda.



08

Funções: erros comuns

Três coisas que fazem a função parecer quebrada quando não está.

o bug clássico

```
function somar(a, b) {  
  console.log(a + b); // só mostra  
}  
let x = somar(2, 3);  
console.log(x); // undefined!  
  
function somarOk(a, b) {  
  return a + b; // devolve  
}  
console.log(somarOk(2, 3)); // 5
```

Esquecer o return

console.log mostra na tela, return entrega o valor para quem chamou. São coisas diferentes.

Declarar e não chamar

function somar() {...} só define. Nada acontece até você escrever somar(2, 3).

Nome no plural do que faz

Dê nomes de ação: calcularMedia, somarNotas, exibirBoasVindas. Facilita ler depois.



Quiz de revisão

Responda sem rodar o código. Depois testamos no console juntos.

1 Qual é o tipo de `true`?

2 O que aparece em `console.log("20" + 5)`?

3 Qual a diferença entre `==` e `===`?

4 Em `[10, 20, 30]`, quanto vale `.length` e o item `[1]`?

5 O que a função devolve se você esquecer o `return`?

6 Como escrever "Oi, Ana!" usando template string?



Bom código!

Riad Younes

Próxima aula: objetos e DOM

Toda dúvida é bem-vinda.

